

Fortifying Robustness in Graph Neural Networks: A Loss Correction Approach to Mitigate Label Noise

I-Chung Hsieh and Cheng-Te Li[✉], *Member, IEEE*

Abstract—Handling noisy labels during neural network training is fundamental, as such noise can skew model optimization, leading to diminished performance. This article zeroes in on node classification using graph neural networks (GNNs), renowned for their adeptness in harnessing both node attributes and edge connections. Nevertheless, GNNs, due to their inherent feature propagation and information amalgamation, are more susceptible to misdirection from label noise than conventional neural networks. While existing robust GNNs offer defenses against noise's adverse effects, they often understate risks posed by noisy labels during training or undervalue the significance of fortifying edges against erroneous aggregation and noisy edges. Addressing these gaps, we introduce a novel robust GNN framework, noise-mitigating GNN (NomiGNN), designed to enhance the robustness of GNNs against label noise. NomiGNN estimates noise distributions and refines loss optimization to counteract noise during training. Moreover, by instituting edge labels for a fresh prediction task, we facilitate learning sample relationships via same-label probabilities, mitigating mis-aggregation from noisy edges and bolstering node classification. Supplementing this, we incorporate pseudoedge labeling and iterative learning to remedy label shortages and estimation inaccuracies. NomiGNN is substantiated by both theoretical rationale and empirical evidence. Experimental evaluations on five real-world graphs highlight NomiGNN's superior resilience against noisy label corruption, outperforming eight benchmark GNN models.

Index Terms—Graph representation learning, label cleaning, loss correction, model robustness, node classification, noise reduction, noisy labels, pseudoedge labeling, robust graph neural networks (GNNs).

NOMENCLATURE

$G(V, A, X)$	Attributed network consisting of a node set V , adjacency matrix A , and feature matrix X .
N	Total number of nodes in the network V .
d	Number of features per node in the feature matrix X .
$\tilde{V}/V$	(Corrupted) Node set.
$\tilde{Y}/Y$	(Corrupted) Label set.
$\tilde{S}/S$	(Corrupted) Edge label set.
c	Number of classes in the label set Y .
GNN	Graph neural network.

y	Node classification label.
$\tilde{A}$	Adjacency matrix with added self-loops ($A + I$).
$\tilde{D}$	Degree matrix corresponding to $\tilde{A}$.
$H^{(l)}$	Hidden state matrix at the l th layer of a GNN.
σ	Activation function used in the neural network.
$W^{(l)}$	Weight matrix of the l th layer of a GNN.
$\tilde{f}/f$	GNN trained on (corrupted) node/edge label set.
$f(y \cdot)/f^e(s \cdot)$	Predicted probability for a given node label y /edge label s .
T/T^e	Noise transition matrix for labels/edge labels.

I. INTRODUCTION

GRAPH neural networks (GNNs) have emerged as a potent tool for various applications, efficiently interweaving learning through neural networks with graph structures [35]. GNNs specifically facilitate the aggregation of latent information from a node's neighborhood via nonlinear mapping features, thereby proficiently leveraging shared messages from related samples for targeted tasks. This capability propels GNNs to exhibit superior performance in (semi-)supervised contexts, such as node classification. In instances where labeled nodes are limited, unlabeled nodes can be inferred through feature propagation leveraging the few available labeled nodes.

Nonetheless, relying on a sparse set of labeled nodes for inferring others signals a potential robustness issue, particularly if the labeled nodes are tainted with noise. In practical applications, noise, which may stem from numerous sources such as insufficient information [10], labeling errors [27], subjective labeling [23], and communication issues between sources [28], is virtually inevitable. Accruing completely clean labels proves to be a challenging task. Additionally, the procurement of high-quality labels often becomes both a financially and temporally expensive endeavor, especially when involving professional annotators [30]. Consequently, frequent labeling of new samples or verification of label accuracy cannot always be achieved.

When addressing label noise in machine learning, a common approach is to adopt the independence assumption, where noisy labels are considered conditionally independent of node features given the true labels [29]. This assumption simplifies the noise modeling process and has been widely adopted in various robust learning frameworks. Previous works on robust

Received 7 August 2024; revised 31 March 2025 and 5 November 2025; accepted 27 January 2026. Date of publication 10 March 2026; date of current version 2 September 2026. This work was supported by the National Science and Technology Council (NSTC) of Taiwan under Grant 113-2221-E-006-201-MY3, Grant 112-2628-E-006-012-MY3, and Grant 113-2634-F-002-007. (Corresponding author: Cheng-Te Li.)

The authors are with the Department of Computer Science and Information Engineering, National Cheng Kung University, Tainan 70101, Taiwan (e-mail: q00117888@gmail.com; chengte@ncku.edu.tw).

Digital Object Identifier 10.1109/TNNLS.2026.3661886

2162-237X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09, 2026 at 12:29:42 UTC from IEEE Xplore. Restrictions apply.

GNNs, such as NT [25] and NRGNN [7], have employed similar assumptions to develop their noise-handling mechanisms. In this article, we follow these established settings while introducing novel techniques to specifically address the unique challenges posed by noise propagation in the graph.

The inherent noise in neural networks poses various challenges for GNNs, including misdirected learning [39], performance decline [12], which cause the inaccurate aggregation, and harmful edge incorporation. GNNs for node classification assume that neighborhoods formed by edges should have consistent labels. Noisy labeling or inappropriate edge allocation can lead to mislabeled nodes, disrupting model updates and corrupting aggregation. If nodes integrate neighbors with inaccurate labels, it can misguide GNNs, causing a significant drop in node classification performance.

To enhance GNN robustness against noise propagation through edges, existing methods integrate self-supervised learning or pseudo-label techniques, generating auxiliary labels and edges to mitigate noise transmission [7], [8], [11], [17]. Yet, while they address edge noise, they often neglect risks from a corrupted training set and cannot fully eradicate harmful edges. For example, NRGNN [7] employs a two-step process including developing new edges and labels via noise-polluted labels and then training GNN. Therefore, corrupted labels and edges cannot eliminate harmful influence in GNNs. A refined approach, exemplified by RSGNN [8], reconstructs edges, maintaining label consistency between nodes, which indirectly reduces harmful edge effects. Nevertheless, RSGNN still resorts to noisy labels of nodes to purify the graph structure. Besides, since link prediction densifies edges through a given graph, the constructed edges still cannot prevent the spread of noise. On the other hand, CGNN [38] uses a graph contrastive learning method to improve the generalization of the model to resist label noise by randomly dropping edges and masking node attributes, and utilizes a label noise correlation method via leverages the homophily. However, there is a concern that leveraging the noise correlation assumption may not adequately address the complexities of label noise in all scenarios. The corrected label loss aims to avoid noise influence, but its effectiveness can vary depending on the noise present in the dataset and pseudolabel quality. Ideally, a robust GNN should both minimize noise impact during training and denoising, and adeptly manage incorrect aggregation and harmful edge effects.

To address identified challenges, we propose a novel GNN framework, noise-mitigating GNN (NomiGNN)¹ displayed in Fig. 1, devised to enhance GNN robustness. NomiGNN, a flexible framework integrated with GNN backbones, leverages robust optimization to avoid training with noisy label and edges through noise distribution estimation. To mitigate the information aggregation via harmful edges, NomiGNN supplants link prediction with a newly defined task *Edge Label Classification* to find homophilous edges, which provide clean and robust supervision signal, for GNN message passing. Additionally, we theoretically prove its effectiveness and low-

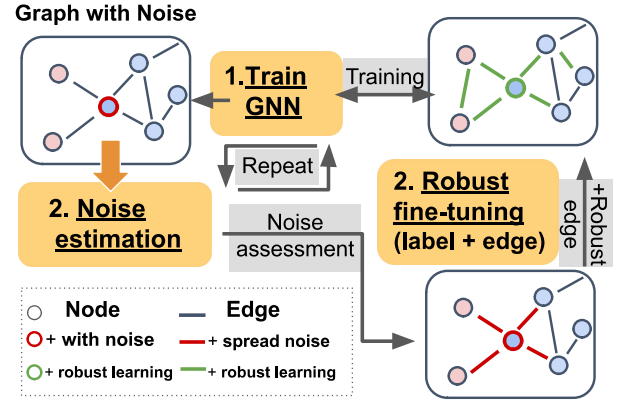


Fig. 1. Overview of the proposed NomiGNN framework: Train GNN f on a noisy graph, and estimate (label/edge) noise distributions. Then, we collect pseudo edge labels and robust fine-tune the model using corrected losses, with these steps repeated to reinforce each other.

misleading rate. We further leverage pseudo edge labeling and iterative learning to counteract label scarcity and imprecise estimation issues. NomiGNN is synergistically trained to discover reliable edges, together with information propagation, via multitask learning, thereby bolstering node classification.

Our contributions can be summarized as follows.

- 1) We illuminate the latent issues of existing robust GNNs in node classification amidst noisy label corruption, which encompasses the detrimental impacts of learning with noise, incorrect aggregation, and harmful edges. To ameliorate these issues, we present a novel GNN framework, NomiGNN, which embarks on robust optimization and refines the graph structure and the GNN learning process.
- 2) A thorough theoretical analysis is provided to elucidate the problem and explicate the functionality of NomiGNN. Further, NomiGNN leverages pseudoedge labeling and iterative learning for label scarcity and imprecise estimation. Beyond robustness, our NomiGNN can augment common GNNs and enhance their node classification by integrating reliable edges and labels probabilistically.
- 3) Through experiments, we showcase the merits of adopting our NomiGNN by conducting baseline comparisons of robustness and accuracy on real-world graphs. Moreover, detailed studies for each model component and various noise scenarios are conducted to elucidate the model's efficacy and resilience.

This article is organized as follows. We first review relevant studies in Section II, followed by presenting the problem statement in Section III. In Section IV, we analyze the problem and describe the proposed NomiGNN. We describe evaluation settings and experimental results in Section V, and conclude this work in Section VI.

II. RELATED WORK

Enhancing the robustness of GNNs has gained substantial traction in the research community. One pioneering work, D-GNN [25], integrated the loss correction method [26] within the GNN context. However, while it mitigated noise effects to

¹Code can be accessed via <https://www.dropbox.com/sh/xo9w79614a9o0xn/AAB9i5m8H28Q10VkJZPgxe55ia?dl=0>

some extent, it overlooked key issues like noise propagation via edges and potential errors in noise estimation. Subsequent research endeavored to offset noise impacts using graph-based data. For instance, Zhang et al. [40] introduced LafAK-D, an approach that leverages clustering of node embeddings to generate auxiliary labels, thereby aiding node classification. UnionNET [21] took a different route, handpicking nodes deemed trustworthy based on edge random walks, while also assuming that neighboring nodes share identical labels. The strategy involved computing average distances between nodes and anchor points within a support set to adjust label probability predictions. On a similar note, PI-GNN [11] employed label propagation to produce auxiliary labels, also incorporating a feature mask to optimize the training for more robust classification.

Recently, a new set of methods has emerged, largely focusing on either curriculum learning or robust representation learning guided by the graph topology. For instance, topological sample selection (TSS) [34] employs learning by prioritizing topologically simpler nodes first, using a metric called Class-conditional Betweenness Centrality to distinguish easy-to-learn clean samples. ERASE [3], conversely, learns error-resilient representations by maximizing interclass separability and intraclass compactness using the Maximizing Coding Rate Reduction principle, complemented by a decoupled label propagation. Finally, R²LP [5] tackles label noise in heterophilous graphs by iteratively reconstructing a more homophilous graph based on node features, upon which standard label propagation is applied. However, a common limitation across these methods is their incomplete handling of noise amplification: TSS and ERASE heavily rely on the integrity of the existing graph structure and do not explicitly address detrimental edges that can amplify noise during GNN aggregation. While R²LP attempts graph reconstruction, its effectiveness is highly contingent on the quality of node features and it still does not explicitly model the noise transition matrix or directly address harmful edges within the original structure, a crucial component for robust aggregation.

These models primarily use unsupervised label estimation to temper noise-induced disruptions in supervised learning, with a reliance on auxiliary labels to diminish dependence on noisy primary labels. Notably, despite their potential to counteract incorrect GNN aggregations, subpar auxiliary labels can backfire, especially when they originate from misclassified labels. Additionally, such labels do not necessarily obstruct deleterious edges that might amplify misinformation.

Shifting the lens to edge noise, several works strive for resilience against corrupted edges, aiming to fine-tune attention to more critical edges, thereby countering misguided aggregations and detrimental edges. For instance, ProGNN [17] employs constraints related to low rank and sparsity to retain untainted edges, simultaneously infusing node features' similarity for crafting a denoised graph. SE-GNN [9], on the other hand, champions self-supervised learning through contrastive learning on a masked graph, aiming for heightened feature and edge similarity. This approach then uses auxiliary labels derived from clustered node embeddings to bolster performance. In a more comprehensive effort, RSGNN

TABLE I
SUMMARY OF RELATED STUDIES ON ROBUST GNNs

	Target	NDist.	SSL	PLabel	NSpread	ELabel
D-GNN [25]	node	✓				
LafAK-D [40]	node		✓	✓		
UnionNET [21]	node		✓	✓		
PI-GNN [11]	node		✓	✓		
ProGNN [17]	edge		✓		✓	
SE-GNN [9]	edge		✓	✓	✓	
RSGNN [8]	edge		✓	✓	✓	✓
NRGNN [7]	node		✓	✓	✓	
CGNN [38]	node		✓	✓		
TSS [34]	node		✓	✓		
ERASE [3]	node		✓	✓		
R ² LP [5]	node		✓	✓	✓	
NomiGNN (ours)	edge+node	✓	✓	✓	✓	✓

[8] simultaneously addresses link prediction and the nexus between edges and label pairs during graph formulation, with the purified graph bolstering node classification robustness. Interestingly, among all models designed for edge noise resilience, only RSGNN directly addresses edges that might hinder node classification, by factoring in the relationship between edges and label pairs. Bridging the gap between label and edge noise mitigation, NRGNN [7] primarily tackles misaggregations. It initially pretrains a GNN using link prediction and node classification to curate a set of auxiliary labels and a refined graph. A subsequent GNN iteration then leverages this new graph for improved label noise robustness predictions. However, even this comprehensive approach falls short in addressing detrimental edges and the perils of noise-tainted learning.

In analyzing the landscape of robust GNNs, we categorize these models based on six distinct dimensions, which we further contrast against our proposed NomiGNN in Table I. The criteria are: 1) *Defense Target*: What specific threats does the GNN seek to defend against, i.e., node-wise or edge-wise classification tasks? 2) *Noise Distribution (NDist.)*: Does the GNN model infer noise distributions, such as transition probabilities from authentic to noisy labels? 3) *Graph Self-Supervised Learning (SSL)*: Is the GNN equipped to assimilate graph data for tasks that require the reconstruction of graph attributes like clustering or link prediction? 4) *Pseudo Labeling (PLabel)*: Does the GNN leverage pseudo labels of nodes or edges to enhance its learning capabilities? 5) *Noise Spreading (NSpread)*: Does the GNN implement strategies to curb the dissemination of noise, potentially via graph reconstruction? 6) *Harmful Edges for Node Classification (ELabel)*: Does the GNN take measures to cleanse or discard edges that can negatively impact node classification? When dissecting the existing body of robust GNNs, it is evident that most primarily target either label or edge noise for node classification. Our work, while centered on label noise, contends that provided edges might not always be optimal for node classification, as alluded to in Section I. Thus, NomiGNN judiciously employs an estimated noise distribution to refine label learning. We eschew traditional link prediction in favor of edge label classification, bolstering GNN aggregation while instilling robustness at the edge level. Diverging from the commonplace use of pseudo labels, pseudo edge labeling in NomiGNN can better aggregate homophilous information for

producing node representations in GNNs, and thus can further minimize prediction inaccuracies. The details are explained in Section IV-C2. Crucially, by intertwining label and edge learning, we ensure edge learning circumvents pitfalls like incorrect aggregations and edges that are detrimental to node classification.

III. PROBLEM STATEMENT

Generally, an attributed network $G = (V, \mathbf{A}, \mathbf{X})$ consists of node set V ($N = |V|$) with node features $\mathbf{X} \in \mathbb{R}^{N \times d}$, where d is the feature number. The N by N adjacency matrix $\mathbf{A}$ depicts edges $(u, v) \in V$ with weights $\mathbf{A}_{uv} > 0$, and $\mathbf{A}_{uv} = 0$ means no relationship between u and v . Given a graph G and the training node set $V_{\text{train}} = (v_1, v_2, \dots, v_k) \in V$ with c class labels, i.e., $Y_{\text{train}} = (y_1, y_2, \dots, y_k) \in \mathcal{Y}^c$, where k is the number of labeled nodes, the node classification task aims to predict labels of unlabeled nodes $V_{\text{test}} = (v_{(k+1)}, v_{(k+2)}, \dots, v_N)$.

Node classification can be tackled via GNNs. The l th layer of L -layer GNN can be depicted as

$$\mathbf{H}^{(l)} = \sigma \left(\tilde{\mathbf{D}}^{(-\frac{1}{2})} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{(-\frac{1}{2})} \mathbf{H}^{(l-1)} \mathbf{W}^{(l-1)} \right) \quad (1)$$

where $l \in \{1, 2, \dots, L\}$, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency with adding identity matrix $\mathbf{I}$ representing self-loop, $\tilde{\mathbf{D}}$ is the degree matrix with $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$, $\mathbf{H}^{(l)}$ is the l -layer's input representation, σ is the activation function (e.g., ReLU), and $\mathbf{W}^{(l)}$ is the l -layer trainable weight matrix. The node features are used to initialize node vectors, i.e., $(\mathbf{H}^{(0)} = \mathbf{X})$, and $\mathbf{H}^{(L)} \in \mathbb{R}^{N \times c}$ derived from normalized activation (e.g., Softmax) in the last layer is the predicted probability matrix. The model is commonly optimized by the cross-entropy loss with the predicted probability $f(y|\cdot)$ and the given labels $y \in Y_{\text{train}}$. We denote $f(y|\cdot)$ as the predicted probability with the model f . The notations are summarized in Nomenclature.

A. Problem: Robust GNNs Against Noisy Node Labels

When training the GNN with the given graph G and the labeled node set V_{train} , we cannot guarantee the complete correctness of node labels Y_{train} , meaning that some labels in Y_{train} could be incorrect. We also cannot know which nodes' labels are incorrect. We redefine V_{train} and $\tilde{V}_{\text{train}}$ as node sets with clean labels Y_{train} and corrupted labels $\tilde{Y}_{\text{train}}$, respectively. Only $\tilde{Y}_{\text{train}}$ is utilized in the GNN model training. Consequently, we aim to build a robust GNN $\tilde{f}$ against the influence of the noise in $\tilde{Y}_{\text{train}}$. We hope that $\tilde{f}$ trained with polluted labels $\tilde{Y}_{\text{train}}$ can still maintain the performance of node classification. That said, we want to the performance produced by $\tilde{V}_{\text{train}}$ -based GNN model $\tilde{f}$ is as close as to that generated by V_{train} -based GNN model f , depicted by

$$\tilde{f}(y|\tilde{V}_{\text{train}}) \approx f(y|V_{\text{train}}) \quad (2)$$

where $y \in \mathcal{Y}^c$.

B. Assumption of Label Noise

The noise randomly occurs in different nodes and their labels. We assume there exists a *noisy label distribution* $p(\tilde{y}|y)$ that represents the probability the training label y is flipped to

$\tilde{y} \in \mathcal{Y}^c$ [22], [24], [36]. We define the noise transition matrix $T \in [0, 1]^{c \times c}$ to describe the label's flipping probability, where $T_{\tilde{y}y} = p(\tilde{y}|y)$ represents the probability that a true label y is flipped to a noisy label $\tilde{y}$. T is asymmetric under probability axioms. The transition matrix will be used in developing the robust GNN algorithm that alleviates the influence of noisy labels.

Our noise model makes a key assumption that the noisy labels $\tilde{y}$ and node features x are conditionally independent given the true labels y

$$p(\tilde{y}|y, x) = p(\tilde{y}|y). \quad (3)$$

This assumption implies that the probability of observing a noisy label depends only on the true label, not on the node features or graph topology. Practically, this means that label flips occur randomly across the graph and are not influenced by node characteristics or neighborhood structure.

This conditional independence allows us to model the noise through a class-conditional noise transition matrix $\mathbf{T}^n \in \mathbb{R}^{C \times C}$, where $T_{\tilde{y}y}^n = p(\tilde{y}|y)$ represents the probability that a true label y is flipped to a noisy label $\tilde{y}$. While this simplifying assumption may not capture all real-world noise patterns (e.g., where noise might correlate with certain features or graph regions), it provides a mathematically tractable framework that has proven effective in practice for noise-robust learning.

IV. METHODOLOGY

This section presents the theoretical foundation and architecture of our NomiGNN framework, which addresses the challenges of learning with noisy labels in graph data through three key components. In Section IV-A, we introduce a robust neural network based on Lemma 1 and demonstrate why this approach alone is insufficient for graph data due to the diffusion of incorrect information across nodes. Section IV-B extends this foundation by deriving Theorem 1 for robust edge label prediction and establishing a methodology for handling edge noise in graph structures. Finally, in Section IV-C, we present the complete NomiGNN architecture, which implements a three-step process: first training a GNN on noisy labels to estimate noise distributions, then applying loss correction through multitask learning for both node and edge labels, and finally enhancing performance via pseudo edge labeling and iterative learning. This comprehensive approach ensures robust learning by simultaneously addressing node label noise and harmful edge propagation while maintaining theoretical guarantees for convergence and performance.

A. Preliminary: Robust Neural Networks

We first review the loss correction approach to robust neural networks. Prevalent methods in this area include the structured approach developed by Patrini et al. [26], which proposes a theorem for robust learning based on the noise transition matrix T . Other approaches, such as generalized cross-entropy (GCE) [41], leverage the noise robustness of mean absolute error (MAE) by adjusting the loss curvature, while symmetric cross-entropy (SCE) [32] employs reverse cross-entropy (RCE) to balance the aggressive learning of

standard Cross-Entropy. Crucially, these methods have not yet been applied to graph data.

The fundamental difference lies in their mechanism: GCE and SCE rely on passive tolerance by reshaping the loss curvature, while the Backward Correction method provides an active, structured approach that explicitly models the noise transition matrix. We select the backward loss correction method [26] due to its superior mechanism, as it is the only known approach that offers a rigorous theoretical guarantee of unbiased loss estimation for asymmetric noise. This structural advantage allows us to construct an unbiased loss estimator, ensuring the corrected loss theoretically equals the clean loss. This strong theoretical foundation, including unbiasedness and the noise-free Hessian property, is crucial for our multitask GNN framework designed for noisy graphs.

Lemma 1 [26]: Given the noise transition matrix T is nonsingular, f is a model for classification with a loss $\mathbf{l}$ for a data sample x . Then, the backward corrected loss is unbiased and defined as

$$\forall x, \mathbb{E}_{\tilde{y}|x} T^{-1} \mathbf{l}(y, f(y|x)) = \mathbb{E}_{y|x} \mathbf{l}(y, f(y|x)). \quad (4)$$

It also implies the equality of the optimal solution, i.e.,

$$\operatorname{argmin}_{f(y|x)} \mathbb{E}_{\tilde{y}|x} T^{-1} \mathbf{l}(y, f(y|x)) = \operatorname{argmin}_{f(y|x)} \mathbb{E}_{y|x} \mathbf{l}(y, f(y|x)). \quad (5)$$

The theorem indicates that the corruption from noisy labels can be corrected by transforming the loss $\mathbf{l}$ with the inverse noise transition matrix T^{-1} . The corrected loss is differentiable, and can be minimized with any off-the-shelf algorithm for back-propagation. The robust learning process makes the model expectedly become robust against the noisy labels, i.e., like training with clean labels. However, the noise transition matrix T is unknown or needs to be estimated. A parametric estimation method can be used to approximate T [26] with some assumptions. One assumption is that there exists the perfect data sample x_j^p that can be found for each class j such that $p(x_j^p) > 0$ and $p(y = j|x_j^p) = 1$. The other assumption is that the noise label probability can be expanded to the sum of products of transition probability and label probability for each class, given by: $p(\tilde{y} = j|x) = \sum_{k=1}^c p(\tilde{y} = j|y = k)p(y = k|x) = \sum_{k=1}^c T_{kj}p(y = k|x)$. With such two assumptions, the transition probability can be reduced by $p(\tilde{y} = j|x_i^p) = T_{ij}p(y = i|x_i^p) + \sum_{k \neq i} T_{kj}p(y = k|x_i^p) = T_{ij}$. Considering the model f satisfying $f(\tilde{y} = j|x) \approx p(\tilde{y} = j|x)$, the noise transition matrix can be estimated by

$$\hat{T}_{ij} = f(\tilde{y} = j|\hat{x}_i^p) \quad (6)$$

where the perfect sample can be chosen in an approximated manner based on the highest predicted probability, $\hat{x}_j^p = \arg \max_x f(\tilde{y} = j|x)$, for every class label $j \in \{1, \dots, c\}$.

1) Discussion: Lemma 1 offers an insightful strategy for defending against label noise in standard neural networks. However, when applying to GNNs, this approach encounters challenges. GNNs not only engage in feature propagation but also inherently weigh the relational connectivity of samples. This connectivity allows for the aggregation of feature information along edges. A pivotal concern arises when an edge

potentially transmits erroneous information, leading to incorrect feature aggregation. Specifically, when GNNs process features associated with noisy labels, such misrepresentations can inadvertently skew predictions for samples that possess similar features. This is further exacerbated by the inherent nature of GNNs, which often assumes that neighboring samples should share similar labels. Consequently, presenting a GNN with an inaccurate label or grouping dissimilar samples within the same neighborhood can easily lead to misaggregations. Thus, solely relying on Lemma 1 for robust learning might not adequately address the challenges posed by the diffusion of incorrect information across nodes in a graph, making it less effective in ensuring GNN robustness against label noise.

B. Robust GNNs

While edges play a key role in passing messages across nodes, to construct a robust GNN, we need to denoise edges. Ideally, to avoid propagating incorrect information across nodes, we should keep all “clean” edges whose end nodes possess the same label, and rule out “harmful” (or noisy) edges, whose end nodes’ labels are inconsistent. To do so, we have to recognize which edges are clean or harmful. GNNs can be utilized to classify clean and harmful edges by treating the task as *edge label prediction*. Here, *edge label* in a graph is defined by the consistency of node labels in an edge, given by: $s(y_u, y_v) = 1$ if $y_u = y_v$; $s(y_u, y_v) = 0$ if $y_u \neq y_v$, where (u, v) is any node pair with labels (y_u, y_v) . Nodes with different labels indicate the unconnected relation. Such a formulation of edge label prediction enjoys a benefit—it is compatible with Lemma 1. Hence, the recognition of clean and harmful edges can be constructed through a robust learning process with proper loss correction.

Due to the label noise, the clean edge label training set S_{train} is corrupted as the noisy edge label training set $\tilde{S}_{\text{train}}$. We can construct $\tilde{S}_{\text{train}}$ by the training set of nodes with known labels $\tilde{Y}_{\text{train}}$. Similarly, the edge e ’s noisy label is denoted as $\tilde{s}$, and the corresponding edge-wise noise transition probability $T^e = p(\tilde{s}|s)$ follows same notations like $\tilde{y}$, where s and $\tilde{s}$ are the clean and noisy labels, respectively, for the edge e . The GNN model for edge label prediction, denoted as f^e , will take the graph G , together with $\tilde{S}_{\text{train}}$ as the input. To mitigate the impact of label noise in the task of node classification, we aim to construct f^e to recognize the ideal edges, i.e., edges with same-label end nodes, for message passing and feature aggregation of GNNs. While the learning of GNN f and f^e could also suffer from label noise, we can extend Lemma 1 to edge label prediction so that a robust learning process can be built. We can reformulate Lemma 1 to Theorem 1 for robust GNN-based edge label prediction against label noise as follows.

Theorem 1: Given the edge-wise noise transition matrix T^e is nonsingular, the edge label prediction model f^e with loss $\mathbf{l}^e$, the corrected loss is unbiased for every node pair (u, v) and defined as

$$\mathbb{E}_{\tilde{s}|u,v} (T^e)^{-1} \mathbf{l}^e(s, f^e(s|u, v)) = \mathbb{E}_{s|u,v} \mathbf{l}^e(s, f^e(s|u, v)). \quad (7)$$

Proof:

$$\begin{aligned}
\mathbb{E}_{\tilde{s}|u,v} [A_{\tilde{s}}] &= \sum_{\tilde{s}} p(\tilde{s}|u,v) A_{\tilde{s}} \\
&= \sum_{\tilde{s}} \left(\sum_s p(\tilde{s}|s) p(s|u,v) \right) A_{\tilde{s}} \\
&= \sum_s p(s|u,v) \sum_{\tilde{s}} p(\tilde{s}|s) A_{\tilde{s}} \\
&= \sum_s p(s|u,v) T^e (T^e)^{-1} L^e(s) \\
&= \sum_s p(s|u,v) L^e(s) \\
&= \sum_s p(s|u,v) \mathcal{I}^e(f(s|u,v)) \\
&= \mathbb{E}_{s|u,v} [\mathcal{I}^e(f(s|u,v))] \quad (8)
\end{aligned}$$

where $A_{\tilde{s}} = (T^e)^{-1} L^e(s)$, $L^e(s) = \mathcal{I}^e(f(s|u,v))$. In (8), we first express the expectation over noisy edge labels $\tilde{s}$ conditioned on node pair (u, v) , and then expand this using the law of total probability to include the true edge label s . The multiplication of T^{e-1} with T^e leads to their cancellation, showing that the corrected loss equals the original loss computed on clean edge labels. This proves that our approach for correcting label noise through edge label loss maintains the expected loss unchanged under the noise model characterized by T^e .

The formula in Theorem 1 implies that the model for edge label prediction can become robust against label noise when applying the loss transformation during optimization.

1) *Discussion:* Intuitively, we can apply Lemma 1 to build a GNN f^e against the noisy edges, and then train the other GNN f with clean edges, i.e., edges with same-label end nodes, predicted by f^e to produce the robust node classification results via Theorem 1. However, implementing such a model flow faces three issues, including a redundant training process, the lack of the training set with clean edge labels, and inaccurate estimation for T and T^e . First, the process is with a higher cost on training two models f and f^e . Although these two models can be combined by multitask learning, which trains one GNN f with two losses $\mathcal{I}$ and $\mathcal{I}^e$ at the same time. However, multitask learning involves a complex training process, and is relatively difficult to optimize [6]. And the arbitrary choice of tasks cannot certainly improve the performance [14]. Second, the size of the training set with noisy edge labels $\tilde{Y}_{\text{train}}$, which is constructed from the known label set Y_{train} , is much smaller than the possible pair amount, i.e., $|V_{\text{train}}|^2 \ll |V|^2$. Because the feature aggregation in GNNs refers to all edges (u, v) for $A_{uv} > 0$, the improvement of robustness could be quite limited due to insufficient edge labels. Third, the noisy labels of edges and nodes could hurt the determination of perfect data sample for each label, which is required in Lemma 1 and Theorem 1, as depicted by $\hat{x}_j^p$ in (6). Hence, semi-perfect samples can subsequently bring inaccurate estimations of T and T^e , which eventually affect the effectiveness of robustness improvement.

C. Proposed NomiGNN Model

Overview: We propose a novel NomiGNN, whose overview is presented in Fig. 1. Our framework comprises a

comprehensive approach to mitigate label noise in GNNs through three interconnected components. First, we derive noise distributions for both node labels (T) and edge labels (T^e) based on Lemma 1 and Theorem 1, utilizing a model f trained with noisy labels to estimate T and subsequently deriving T^e through the relationship between clean edges and node labels. Second, we implement a multitask learning paradigm that incorporates noise distribution transformations via T^{-1} and T^{e-1} into the loss functions $\mathcal{I}$ and $\mathcal{I}^e$, simultaneously addressing node and edge classification while leveraging their theoretical interdependence to ensure coordinated optimization. Third, we enhance model robustness through iterative refinement, employing pseudo edge labeling to address edge label scarcity and progressively refining noise transition matrices to reduce estimation errors. Overall, our NomiGNN framework provides a theoretically grounded approach that simultaneously addresses node label noise and harmful edge propagation by establishing clear relationships between node and edge label probabilities, thereby ensuring robust learning with provable guarantees and significantly improving classification accuracy in the presence of noisy graph data.

1) *Noise Estimation on Node and Edge Labels:* In the first step, we review the relationship between T and T^e based on Theorems 1 and 2. We initiate the process by leveraging their relationship between T and T^e , linking node label predictions to edge label predictions. Given any node pair (u, v) , the noise transition probability of an edge label, $p(\tilde{s}|s)$, can be denoted as $p(\tilde{s}(\tilde{y}_u, \tilde{y}_v)|s(y_u, y_v))$. Here, $s(y_u, y_v) : \mathcal{Y} \times \mathcal{Y} \rightarrow \{0, 1\}$ and $\tilde{s}(\tilde{y}_u, \tilde{y}_v) : \mathcal{Y} \times \mathcal{Y} \rightarrow \{0, 1\}$ are indicator functions that map the node label pairs to the edge label consistency state s and $\tilde{s}$, respectively. We define $\mathcal{S}_{s,\tilde{s}} \equiv \{(y_u, y_v, \tilde{y}_u, \tilde{y}_v) \in \mathcal{Y}^4 \mid s(y_u, y_v) = s \text{ and } \tilde{s}(\tilde{y}_u, \tilde{y}_v) = \tilde{s}\}$ as the set of all node label combinations that result in the edge label transition $s \rightarrow \tilde{s}$. Specifically, assuming the noise transition probability is independent of each node, the probability can be computed as

$$\begin{aligned}
T_{\tilde{s}s}^e &= p(\tilde{s}|s) \\
&= \sum_{(y_u, y_v, \tilde{y}_u, \tilde{y}_v) \in \mathcal{S}_{s,\tilde{s}}} p(\tilde{y}_u|y_u) p(\tilde{y}_v|y_v) \\
&= \sum_{(y_u, y_v, \tilde{y}_u, \tilde{y}_v) \in \mathcal{S}_{s,\tilde{s}}} T_{y_u \tilde{y}_u} T_{y_v \tilde{y}_v} \quad (9)
\end{aligned}$$

where $s, \tilde{s}$ signifies label consistency states (i.e., $y_u = y_v$ or $y_u \neq y_v$). Building on this, the node-wise noise transition matrix, $\hat{T}$, deduced from (6), aids in determining values for the edge-wise matrix, $\hat{T}^e$, given by

$$\hat{T}_{\tilde{s}s}^e = \sum_{(y_u, y_v, \tilde{y}_u, \tilde{y}_v) \in \mathcal{S}_{s,\tilde{s}}} \hat{T}_{y_u \tilde{y}_u} \hat{T}_{y_v \tilde{y}_v}. \quad (10)$$

With the utilization of (10), direct training of model f on $\tilde{Y}_{\text{train}}$ to obtain T^e becomes redundant. Consequently, the initial training phase for model f focuses on node classification, preceding the integrated multitask learning for both node and edge labels.

2) *Multitask Learning for Node and Edge Labels:* During this multitask learning phase, the interplay between node and edge labels is fortified by predicting $p(\tilde{s}|s)$ based on $p(\tilde{y}|y)$. This enhances the model's efficiency and fortifies its capability

to optimize node classification. It is crucial to recognize that predictions via model f are categorized into pointwise (for node labels) and pairwise (for edge labels) aspects. The prediction probability $p(y|v)$ relates to label y for a specified node v . Thus, for any node pair u and v , probabilities $p(y_u|u)$ and $p(y_v|v)$ can be manipulated to infer edge label probabilities, represented as

$$p(s|u, v) = \sum_{\substack{y_u, y_v \in \mathcal{Y} \\ \text{s.t. } s(y_u, y_v) = s}} p(y_u|u) p(y_v|v) \quad (11)$$

where $s \in \{1, 0\}$ is the edge label consistency state. Given a specific model f for node classification, edge label probability can be predicted via

$$f^e(s|u, v) = \sum_{\substack{y_u, y_v \in \mathcal{Y} \\ \text{s.t. } s(y_u, y_v) = s}} f(y_u|u) f(y_v|v). \quad (12)$$

The training process of NomiGNN can be summarized into two steps: 1) we train f with noisy node labels $\tilde{Y}_{\text{train}}$ for estimating T and deriving $\hat{T}^e$ via $\hat{T}$ in (10); and 2) We fine-tune f for the classifications of node and edge labels via multitask learning with corrected losses. Specifically, applying Lemma 1 and Theorem 1, the model fine-tuning f with $I(f(y|v), \tilde{y})$ and $I^e(f^e(s|u, v), \tilde{s})$ under loss transformation $\hat{T}^{-1}$ and $\hat{T}^{e-1}$, where $f^e(s|u, v)$ is derived from (12) with $f(y_u|u)$ and $f(y_v|v)$. This dual-phase training process ensures that the model first establishes a foundational understanding of the noise distributions within noisy labels, and then progresses to a more nuanced multitask learning mechanism.

a) *Node label loss is bounded by edge label loss:* Here we provide a theoretical foundation for our multitask learning approach that incorporates both node and edge label losses. We demonstrate that this design is not merely a heuristic choice but rather a mathematically principled framework that offers provable benefits. Specifically, we establish that optimizing the edge label loss provides an upper bound guarantee on node label loss, thereby creating a relationship between these two learning objectives that enhances the overall performance of the model.

We explain the benefits of the NomiGNN model design by analyzing how the classifications of node and edges influence each other. Consider that I and I^e as the cross-entropy losses for node label y and edge label $s(y_u, y_v)$ of node pair (u, v) . The optimization process aims to decrease the cross-entropy losses $-\log(p(y|v))$ and $-\log(p(s(y_u, y_v)|u, v))$. That said, both $p(y|v)$ and $p(s(y_u, y_v)|u, v)$ will be increased. Therefore, the model optimization aims to increase $1 - \sum_{y_u \neq y_v} p(y_u|u) * p(y_v|v)$ and $\sum_{y_u = y_v} p(y_u|u) * p(y_v|v)$.

The former (i.e., $s = 0$) implies that the sum of the product $p(y|u) * p(y|v)$ for $y \in \{1, \dots, c\}$ should be low; that is, preventing high probabilities of both u and v from being predicted as the same class label. So, given $y_v = i$ for node v , if there exists $c - 1$ nodes $\{u_k\}_{k=1}^{c-1}$ with different labels $y_v \notin \{y_{u_k}\}_{k=1}^{c-1}$, and the predicted probability is dominated by the belonging class, the optimization of edge labels for $y_v \neq y_{u_k}$ would make $p(y_v = i|v)$ as high as possible. Note that the conditions are easy to achieve because we prepare at least one perfect sample of each class label for node classification.

For the latter (i.e., $s = 1$, $y_u = y_v = y$), we can establish the relationship between loss and predicted probability via

$$\begin{aligned} p(s(y_u, y_v) = 1|u, v) &= \sum_y p(y|u) * p(y|v) \\ &\leq c \cdot p(y_u = y^*|u) p(y_v = y^*|v) \\ &\leq c \cdot \min(p(y_u = y^*|u), p(y_v = y^*|v)) \\ &\Rightarrow \max(-\log(p(y_u = y^*|u)), -\log(p(y_v = y^*|v))) \\ &\leq c^{-1} \cdot -\log(p(s(y_u, y_v) = 1|u, v)) \\ &\Rightarrow -\log(p(y_u = y^*|u)), -\log(p(y_v = y^*|v)) \\ &\leq c^{-1} \cdot -\log(p(s(y_u, y_v) = 1|u, v)) \end{aligned} \quad (13)$$

where c is the class number and $y^* = \arg \max_y p(y|u)p(y|v)$ is the label corresponding to the maximum probability product. The inequality shows that because the sum of probability products is bounded by its dominated probability $y = y^*$, the loss of the edge label prediction is the upper bound of the loss of node label classification when y^* is the correct label. Accordingly, the minimization of the edge label prediction loss leads to a much lower loss of node classification, meaning that the performance of node classification can be further improved.

Beyond mitigating noise aggregation as demonstrated earlier, this theoretical result provides a justification for NomiGNN's edge classification component. The analysis reveals two advantages: 1) it establishes a mathematically optimization when jointly training tasks; and 2) it demonstrates that by minimizing the edge label loss, we effectively establish an upper bound on node classification loss, leading to improved performance. This mathematical relationship between node and edge label losses represents a key innovation of NomiGNN that distinguishes it from previous robust GNN approaches.

3) Pseudoedge Labeling and Iterative Learning:

a) *Pseudo edge labeling:* For the lack of edge labels $\tilde{S}_{\text{train}}$, PLabel [19] is a solution to alleviate its impact. PLabel expands the training label set by adding pseudo labels to instances with high predicted probability values. While most existing GNN methods like Self-Training [20] and NRGNN [7] focus on pseudo node labeling, pseudo edge labeling is not well investigated. For general classification tasks, in fact, the reduction of noisy label rate can be theoretically guaranteed for learning in a *pairwise* manner [33]. Given the edge label's predicted probability $f(s|u, v)$ of a node (u, v) , we utilize a threshold τ to collect the pseudo edge labels of node pair (u, v) for $s = 0$ (and $s = 1$) if the predicted probability $p(s|u, v)$ is low (and high). More types of edge inconsistency (i.e., $c \cdot (c - 1)$) would amplify the tolerance of the wrong prediction for a randomly chosen pair of nodes. In other words, more class labels can increase the performance of predicting the label inconsistency (i.e., $s = 0$) of a node pair. We will conduct experiments to verify that adding pseudo edge labels brings more performance improvement than adding pseudo node labels. In practice, the number of all possible node pairs $|V|^2$ could be very large. Considering the efficiency issue, we utilize only edges (u, v) with $\mathbf{A}_{uv} > 0$ and $f(s|u, v) > \tau$ for

Algorithm 1 Proposed NomiGNN Algorithm

Input: $\mathbf{G} = (V, \mathbf{A}(E), \mathbf{X})$, noise training label $\tilde{Y}_{train}$, GNN model f , edge label threshold τ , estimation training epoch N_{train} , robust fine-tuning epoch N_{FT} , iteration number N_i

while iteration $\leq N_i$ **do**

 #Noise estimation phase

- Train f via $\mathbf{G}$ and $\tilde{Y}_{train}$ within N_{train}
- Estimate noise transition matrix $\hat{T}$ [see (6)] and $\hat{T}^e$ [see (10)]
- Construct edge label set $\tilde{S}_{train}$ via $f(s|u, v)$ and threshold τ for any pair u, v

 #Robust fine-tuning phase

- Derive $\hat{T}^{-1}$ and $\hat{T}^{e-1}$
- Fine-tune f via $\mathbf{G}$, $\tilde{Y}_{train}$ and $\tilde{S}_{train}$ with loss transformations [see (4) and (7)] within N_{FT}

return f

pseudo edge labeling of the consistency case (i.e., $s = 1$). The negative node pairs (i.e., $s = 0$) are randomly generated from $f(s|u, v) \leq \tau$ with the size as same as the positive case.

b) *Iterative learning*: For the estimations of T and T^e , it is hard to diminish the error that comes from the estimated perfect sample $\hat{v}^p = \arg \max_v f(y|v)$, that is $1 - \max(f(y|\hat{v}^p))$. We reduce the estimation error by reestimating T and T^e after updating GNN. In other way, the iterative process is based on the theory that each subsequent model f^{k+1} provides a more accurate noise model approximation than its predecessor f^k for iteration k . The iterative learning cycles between two main steps: noise estimation and robust fine-tuning. Initially, the model f is trained on a noisy labeled dataset to gauge the noise distributions T and T^e . Following this, the model is fine-tuned using the corrected loss functions for the estimated noise, enhancing its robustness.

4) *NomiGNN Algorithm*: According to the aforementioned analysis, we summarize the framework of NomiGNN in Algorithm 1. The framework has two components including noise estimation and robust fine-tuning. First, given a GNN $f : \mathbf{G} \mapsto [0, 1]^c$, training f on $\tilde{Y}_{train}$ with cross-entropy loss $\mathcal{L}$. Then, we select the samples with maximum predicted probability to estimate the noise transition matrix $\hat{T}$ [see (6)] and $\hat{T}^e$ [see (10)]. Besides, the construction of the edge label set $\tilde{S}_{train}$ depends on the criteria $f(s|u, v) > \tau$ or $f(s|u, v) \leq \tau$ for any node pair (u, v) , where the threshold τ is a scalar to ensure the quality of pseudoedge label. In the step of robust fine-tuning, we fine-tune the model f with on $\tilde{Y}_{train}$ and $\tilde{S}_{train}$ with cross-entropy transformed loss $T^{-1}\mathcal{L}$ [see (4)] and $T^{e-1}\mathcal{L}^e$ [see (7)] for the robust learning, respectively. Note that while the matrix inversion for T^{-1} has $\mathcal{O}(c^3)$ complexity, this cost is minimal in GNN benchmarks where c is typically much smaller (e.g., averaged c over all datasets is 8) than the number of nodes and edges. Furthermore, $\hat{T}$ is updated only once per N_{train} epochs, and the overall training efficiency is dominated by GNN message passing over the graph structure. Finally, we repeat these two steps until achieving the given maximum epoch or early stop settings.

TABLE II

STATISTICS OF GRAPH DATASETS. NOTE THAT THE LAST COLUMN “EDGE LABEL RATE” REFLECTS THE DEGREE OF HOMOPHILY

Dataset	#Nodes	#Features	#Edges	Density	#Labels	Edge label rate
Cora	2485	1433	10138	0.003	7	0.8041
Citeseer	2110	3703	7336	0.0033	6	0.7364
Polblogs	1222	1490	33428	0.0448	2	0.9058
ACM	2464	1870	26256	0.0087	3	0.8208
DBLP	2591	334	7056	0.0021	4	0.7988
Pubmed	19717	500	88676	0.0005	3	0.8024
Ogbn-arxiv	169343	128	2315598	0.0002	40	0.6542

TABLE III

STATISTICS OF HETEROPHILIC GRAPH DATASETS. NOTE THAT THE LAST COLUMN “EDGE LABEL RATE” REFLECTS THE DEGREE OF HOMOPHILY

Dataset	#Nodes	#Features	#Edges	Density	#Labels	Edge label rate
Wisconsin	251	1703	499	0.0159	5	0.3571
Chameleon	2277	2325	36101	0.0139	5	0.2569
Squirrel	5201	2089	217073	0.0160	5	0.2322

V. EXPERIMENTS

A. Experimental Settings

1) *Datasets*: We use five well-known graph datasets for the experiments, including Cora [13], Citeseer [13], Polblogs [1], ACM [2], DBLP [2], Pubmed [13], and Ogbn-arxiv [16]. The data statistics are shown in Table II. Note that, we denote the ratio of the label consistency as the edge label rate.

In addition to these homophilic graphs, we also include heterophilic graph datasets to evaluate our method’s performance in low-homophily settings. These datasets include Wisconsin, Chameleon, and Squirrel [43], which have much lower edge label rates (homophily) compared to the aforementioned datasets, as shown in Table III. Heterophilic graphs represent scenarios where connected nodes tend to have different labels, creating a more challenging learning environment for GNNs that typically rely on homophily assumptions.

2) *Competing Baselines*: We aim to examine whether incorporating the proposed NomiGNN into each of the following eight GNN models can improve the performance of node classification. These incorporated GNNs include GCN [18], GraphSAGE (GS) [15], GIN [37], GAT [31], GCNII [4], LafAK-D [40], NRGNN [7], ProGNN [17], RSGNN [8], TSS [34], and ERASE [3]. The first five are typical GNNs, and the remaining are robust GNNs. We compare each GNN model (e.g., GCN) with its corresponding noise-mitigating version by appending “Nomi” as the prefix in every model’s name (e.g., NomiGCN). In addition, to further evaluate NomiGNN’s robustness under extreme noise conditions and compare it with loss correction approaches, we also include baselines that apply the loss correction technique from Patrini et al. [26] to various GNN models, including Correct-GCN, Correct-RSGNN, and Correct-NRGNN. These baselines directly correct the loss function using an estimated noise transition matrix. We evaluate these methods on datasets Cora, Citeseer, Polblogs, ACM, and DBLP with noise rates ranging from 0.0 to 0.9 across 10 different levels.

3) *Settings*: To generate the training set of nodes with noisy labels, we randomly change the label of each node into a different one. The selection of noisy (perturbed) label

TABLE IV
ACCURACY SCORES OF NODE CLASSIFICATION ON CORA, CITESEER, AND POLBLOG. “↑” INDICATES IMPROVEMENT IN PERCENTAGE

	Cora						CiteSeer						Polblog					
Noise Rate	0.0	0.1	0.2	0.3	0.4	↑(%)	0.0	0.1	0.2	0.3	0.4	↑(%)	0.0	0.1	0.2	0.3	0.4	↑(%)
GCN	0.7953	0.7570	0.7077	0.6463	0.5986		0.6616	0.6410	0.6076	0.5690	0.5105		0.9501	0.9073	0.8417	0.7312	0.6669	
NomiGCN	0.8262	0.8041	0.7659	0.7303	0.6741	8.44*	0.7149	0.6782	0.6446	0.6020	0.5673	7.27	0.9501	0.9507	0.9019	0.7699	0.7026	4.34*
GS	0.7371	0.6949	0.6466	0.5682	0.5394		0.6371	0.6088	0.5555	0.5215	0.4646		0.9379	0.9275	0.9190	0.8531	0.7416	
NomiGS	0.8172	0.7922	0.7537	0.7225	0.6731	17.97*	0.7211	0.6914	0.6486	0.5983	0.5662	15.72*	0.9446	0.9432	0.9411	0.9314	0.7831	3.75*
GIN	0.6980	0.6463	0.6039	0.5318	0.4758		0.6356	0.5944	0.5394	0.4853	0.4184		0.9158	0.8576	0.7942	0.7414	0.6415	
NomiGIN	0.7041	0.6618	0.6042	0.5458	0.4960	1.90*	0.6444	0.6083	0.5493	0.4956	0.4393	2.39*	0.9175	0.9013	0.8865	0.8781	0.8565	12.39*
GAT	0.8052	0.7916	0.7555	0.7040	0.6644		0.6829	0.6627	0.6390	0.6115	0.5554		0.9524	0.9465	0.9496	0.9341	0.8433	
NomiGAT	0.8200	0.7916	0.7641	0.7128	0.6713	1.05*	0.6911	0.6788	0.6420	0.6295	0.5712	1.94*	0.9528	0.9461	0.9513	0.9435	0.9426	2.39*
GCNII	0.8261	0.7924	0.7735	0.7452	0.7061		0.6992	0.6915	0.6807	0.6600	0.6278		0.4962	0.5007	0.5051	0.5137	0.5085	
NomiGCNII	0.8339	0.8083	0.7860	0.7583	0.7253	1.78*	0.7015	0.6944	0.7112	0.6972	0.6514	2.87*	0.9563	0.8644	0.7811	0.7741	0.7192	62.23*
LafAK-D	0.8026	0.7759	0.7335	0.6745	0.6312		0.6810	0.6488	0.6133	0.5682	0.5137		0.9550	0.9521	0.9463	0.9103	0.81015	
NomiLafAK-D	0.8224	0.7948	0.7696	0.7430	0.6813	5.54*	0.7351	0.6933	0.6519	0.6187	0.5840	11.05*	0.9487	0.9530	0.9467	0.9480	0.8618	2.04*
NRGNN	0.8308	0.8187	0.8005	0.7865	0.7588		0.7136	0.6999	0.6900	0.6799	0.6408		0.9468	0.9233	0.9055	0.8052	0.7205	
NomiNRGNN	0.8451	0.8231	0.8138	0.7918	0.7729	1.29*	0.7393	0.7235	0.7006	0.6915	0.6553	2.51*	0.9518	0.9352	0.9204	0.8397	0.7559	2.36*
NRGNN (Pair)†	0.8188	0.7989	0.7718	0.7090	0.6247		0.7055	0.6854	0.6632	0.6145	0.5458		0.9468	0.9233	0.9055	0.8052	0.7206	
NomiNRGNN (Pair)‡	0.8339	0.8245	0.8098	0.7523	0.6435	3.49*	0.7262	0.7092	0.6879	0.6399	0.5654	3.42*	0.9553	0.9514	0.9409	0.8940	0.7657	6.58*
ProGNN	0.8229	0.8090	0.7816	0.7493	0.7102		0.6738	0.6658	0.6384	0.6034	0.5505		0.9518	0.9486	0.9099	0.8528	0.7429	
NomiProGNN	0.8233	0.8129	0.7895	0.7680	0.7426	1.63*	0.6951	0.6849	0.6555	0.6275	0.5886	3.82*	0.9519	0.9502	0.9306	0.8810	0.7889	2.19*
RSgNN	0.8168	0.7955	0.7664	0.7338	0.6973		0.7188	0.7074	0.6917	0.6805	0.6450		0.9024	0.8367	0.8171	0.7277	0.6604	
NomiRSgNN	0.8233	0.8017	0.7824	0.7472	0.7171	1.62*	0.7201	0.7175	0.7088	0.7017	0.6643	2.00*	0.9264	0.9386	0.9339	0.9276	0.8973	17.23*
CGNN	0.8010	0.7649	0.7124	0.6640	0.6110		0.6624	0.6347	0.5961	0.5592	0.4993		0.9375	0.9227	0.8851	0.8056	0.6993	
NomiCGNN	0.8219	0.7842	0.7445	0.6954	0.6404	4.42*	0.7019	0.6636	0.6128	0.5717	0.5345	3.71*	0.9513	0.9457	0.9319	0.8565	0.7399	5.73*
TSS	0.8033	0.7540	0.7098	0.6419	0.5734		0.7044	0.6736	0.6209	0.5818	0.5190		0.9039	0.8885	0.8763	0.8865	0.6350	
NomiTSS	0.8255	0.7827	0.7475	0.7042	0.6273	6.20*	0.7150	0.6742	0.6404	0.6214	0.5545	3.68*	0.9182	0.9305	0.9059	0.8885	0.6912	3.75*
Erase	0.7875	0.7886	0.7666	0.7576	0.7221		0.6355	0.6339	0.5821	0.5580	0.5449		0.5767	0.5699	0.5804	0.6115	0.6104	
NomiErase	0.7989	0.8009	0.7932	0.7859	0.7556	2.97*	0.6480	0.6554	0.6282	0.6131	0.6165	7.26*	0.8460	0.8474	0.8460	0.8425	0.7440	40.17*
↑(%)	2.40	2.77	3.85	6.01	6.26		3.68	3.76	4.58	5.64	7.85		13.27	13.55	12.87	14.81	16.56	

is based on uniform distribution [7]. We focus on uniform label flipping as it is a common strategy of producing noisy labels in evaluating robust GNNs [7], [8], [17], [25], [40]. Different noise rate means that different percentages of nodes’ labels are perturbed. Based on practical instances of label noise [10], [23], [27] and the common experimental settings [7], we set the noise rate in $\{0, 0.1, 0.2, 0.3, 0.4\}$. Besides the uniform noise setting, we also conduct the experiments with pair noise, where labelers are assumed to make mistakes only within the most similar pair classes. In this setting, labels have a probability p to flip to their pair class, which simulates more realistic annotation errors where confusion occurs between semantically similar classes. This specialized noise pattern provides an important test for our method’s robustness against structured rather than random noise.

We follow existing studies [7], [8], [17] to extract the largest component of the graph for the experiments, and split the nodes with ratios 10%–10%–80% for the training, validation, and testing, respectively. For the hyperparameter settings of the incorporated GNN models, we refer to their original papers and follow their tuning strategies. We set the maximum epoch number of noise estimation training $N_{\text{train}} = 200$, the maximum epoch number of robust fine-tuning as $N_{\text{FT}} = 100$, and the number of iterative learning as $N_c = 20$. The threshold for the pseudo edge labeling is selected by the validation set with $\tau \in \{0.1, 0.2, \dots, 0.9\}$. We repeat every experiment 10 times with different random seeds, and report the average scores of accuracy, which is the evaluation metric.

B. Main Results

The performance comparison of node classification in terms of accuracy scores is exhibited in Tables IV and V, in which the asterisk * at baseline improvement means statistically significant by Wilcoxon Signed-Rank Test, where the noise rate of node labels is varied with $\{0.0, 0.1, 0.2, 0.3, 0.4\}$. Moreover, in Table IV, we present results for both uniform noise and pair noise settings. The pair noise rows [NRGNN (Pair) and

NomiNRGNN (Pair)] show how our method performs when noise is structured between semantically similar classes. The results on the large scaled graph (Pubmed and Ogbn-arxiv) are shown in Table VI, where OOM means Out-Of-Memory. Note that few GNNs require computational operations (e.g., the attention mechanism in GAT needs $O(\text{neighbor}^2 \cdot d)$, and SSL via edge reconstruction of robust GNNs requires $O(N^2 \cdot d)$ or $O(\text{edge} \cdot d)$), which cause the constraints of GPU memory. We can have several key findings and insights.

- 1) *Model Robustness*: NomiGNN significantly improves the robustness of most GNN models against label noise across various datasets. This is evident by the consistent performance improvement of models incorporated with NomiGNN across different noise rates, as verified by Wilcoxon Signed-Rank Test (indicated by * in Tables IV, V, and VII).
- 2) *Performance Improvement*: When examining the Improvement $\uparrow(\%)$ column, it is clear that NomiGNN’s incorporation consistently improves the performance across various GNN models. For instance, the improvement is as high as 17.97% for NomiGS on the Cora dataset, with the average improvement around 5%–7% for most GNN models.
- 3) *Consistent Performance across Datasets*: This robustness is not just limited to a single dataset. The performance boost of using NomiGNN is observed consistently across multiple datasets.
- 4) *Increasing Resilience to Noise*: As the noise rate increases, there is a consistent decline in the accuracy for all models, which is expected. However, models incorporated with NomiGNN tend to degrade at a slower pace. This suggests that NomiGNN provides a protective mechanism against label noise.
- 5) *Exceptional Models*: Among all, NomiGCNII for the Polblog dataset showcases an impressive improvement of 62.23%, making it a significant outlier. That is due to a smaller number of nodes but a larger number of edges

TABLE V
ACCURACY OF NODE CLASSIFICATION ON ACM AND DBLP

Noise Rate	ACM						DBLP					
	0.0	0.1	0.2	0.3	0.4	$\uparrow(\%)$	0.0	0.1	0.2	0.3	0.4	$\uparrow(\%)$
GCN	0.9068	0.8949	0.8796	0.8390	0.7532		0.7800	0.7400	0.7106	0.6488	0.5856	
NomiGCN	0.9166	0.9041	0.8828	0.8481	0.7661	1.03*	0.7922	0.7548	0.7283	0.6490	0.5949	1.56*
GS	0.9076	0.8867	0.8647	0.8262	0.7357		0.8025	0.7604	0.7153	0.6587	0.6095	
NomiGS	0.9070	0.8914	0.8768	0.8450	0.7915	2.15*	0.8105	0.7795	0.7225	0.6625	0.6206	1.39*
GIN	0.9037	0.8669	0.8224	0.7466	0.6425		0.7831	0.7491	0.6991	0.6492	0.5758	
NomiGIN	0.9022	0.8673	0.8346	0.7812	0.7541	3.95*	0.8047	0.7752	0.7612	0.7385	0.7042	9.48*
GAT	0.8985	0.8895	0.8799	0.8550	0.7786		0.7776	0.7593	0.7205	0.6783	0.6311	
NomiGAT	0.8985	0.9006	0.8831	0.8645	0.7900	0.82*	0.7892	0.7662	0.7377	0.6825	0.6425	1.44*
GCNII	0.9007	0.8940	0.8897	0.8789	0.8341		0.7468	0.7171	0.7070	0.6781	0.6218	
NomiGCNII	0.9077	0.8961	0.8893	0.8797	0.8462	0.49*	0.8184	0.8015	0.7748	0.7368	0.6894	10.09*
LafAK-D	0.9068	0.8888	0.8725	0.8252	0.7210		0.8091	0.7818	0.7491	0.7006	0.6480	
NomiLafAK-D	0.9069	0.8943	0.8857	0.8706	0.7996	5.43*	0.8246	0.8057	0.7570	0.7247	0.6471	1.88*
NRGNN	0.8873	0.8811	0.8802	0.8790	0.8406		0.8129	0.8076	0.7889	0.7572	0.7281	
NomiNRGNN	0.9046	0.9061	0.9020	0.8874	0.8600	2.10*	0.8215	0.8154	0.7927	0.7612	0.7359	0.82*
NRGNN (Pair) [†]	0.8874	0.8864	0.8785	0.8553	0.7915		0.7853	0.7838	0.7557	0.7110	0.6427	
NomiNRGNN (Pair) [†]	0.9113	0.9081	0.8971	0.8605	0.7871	2.49*	0.8401	0.8350	0.8150	0.7780	0.6890	7.19*
ProGNN	0.8926	0.8715	0.8358	0.7786	0.6742		0.7936	0.7832	0.7637	0.7443	0.6850	
NomiProGNN	0.9027	0.8775	0.8429	0.7840	0.6879	1.04*	0.8118	0.8028	0.7878	0.7721	0.7375	3.77*
RSGNN	0.9043	0.9008	0.8978	0.8804	0.8327		0.8124	0.8028	0.7857	0.7540	0.7231	
NomiRSGNN	0.9058	0.9031	0.8982	0.8943	0.8876	1.65*	0.8191	0.8173	0.7939	0.7695	0.7453	1.73*
CGNN	0.9081	0.8953	0.8774	0.8326	0.7442		0.8141	0.7816	0.7466	0.7019	0.6439	
NomiCGNN	0.9081	0.8953	0.8774	0.8438	0.7781	1.05*	0.8161	0.7818	0.7466	0.7023	0.6439	0.05
TSS	0.9008	0.8396	0.7695	0.6783	0.5890		0.8056	0.7270	0.6927	0.5668	0.5041	
NomiTSS	0.9021	0.8458	0.7831	0.6936	0.6050	1.52*	0.8138	0.7434	0.7106	0.5991	0.5591	4.49*
Erase	0.5220	0.5565	0.6167	0.6290	0.6027		0.6693	0.6623	0.6547	0.6183	0.5496	
NomiErase	0.7173	0.7984	0.7957	0.8092	0.7807	33.62*	0.7877	0.7865	0.7867	0.7776	0.7362	23.26*
$\uparrow(\%)$	0.46	0.75	0.93	1.96	5.33		2.16	2.84	2.97	3.56	5.54	

in Polblog. Then NomiGNN can help GCNII to filter out more disruptive edges, thereby reducing the noise and confusion in the learning process of GCNII. This could indicate that certain GNN models, when combined with NomiGNN, become particularly robust for specific datasets.

- 6) *Relative Performance*: While NomiGNN boosts the performance of each model, the relative performance of the models remains mostly unchanged. For instance, if a model performs best without NomiGNN, it is likely to remain at the top even after the NomiGNN incorporation. In conclusion, the incorporation of NomiGNN significantly enhances the resilience and performance of existing GNN models against label noise across multiple datasets. These consistent improvements, both in terms of accuracy and resilience to noise, make a strong case for the utility and effectiveness of NomiGNN in real-world noisy scenarios.
- 7) *Homophily Influence*: The study shows no straightforward relationship between the edge label rate and the performance improvements achieved by NomiGNN across different datasets, as datasets with both low and high edge label rates exhibited varying accuracy gains. These findings indicate that NomiGNN's effectiveness across different graph structures is influenced by its architecture and mechanisms, not just the edge label rate.
- 8) *Robustness to Different Noise Patterns*: Examining the pair noise experiments [NRGNN (Pair) and NomiNRGNN (Pair)], we observe that NomiGNN consistently improves performance even when noise follows a structured pattern between similar classes. The improvement

percentages for pair noise (3.49%, 3.42%, and 6.58% on Cora, CiteSeer, and Polblog, respectively) demonstrate that our method's robustness. This confirms NomiGNN's effectiveness in real-world scenarios like annotation errors.

- 9) *Benefits of Combining Loss Correction with Edge Classification*: Examining Table IV, we observe that while Correct-GCN improves over standard GCN through loss correction alone, NomiGCN consistently achieves even better performance by combining loss correction with edge classification. As noise rates increase, NomiGCN outperforms Correct-GCN. These results clearly demonstrate that while loss correction provides a foundation for handling noisy labels, the integration with edge classification in NomiGCN creates a more powerful framework that leverages graph structure to achieve greater robustness across different datasets and noise conditions.

- 1) *Comparing to Loss-Correction Baselines*: The performance comparison on node classification between NomiGNN and loss-correction baselines (Correct-GNN variants by applying only Patrini et al. [26] method) is exhibited in Table VII. Across Cora, Citeseer, Polblog, ACM, and DBLP with noise rates spanning $0.0 \rightarrow 0.9$, every NomiGNN variant systematically outruns its "Correct-" counterpart in the great majority of settings, and the edge widens as corruption intensifies. From 0.5 to 0.9 noise, NomiGNN is superior in well over 90% of model-dataset-rate cells, whereas the margin is smaller ($\approx 80\%$) when noise is light (0.0–0.4). The pattern is consistent across different backbones (GCN/NRGNN/RSGNN): accuracy degrades more slowly for NomiGNN because it does not

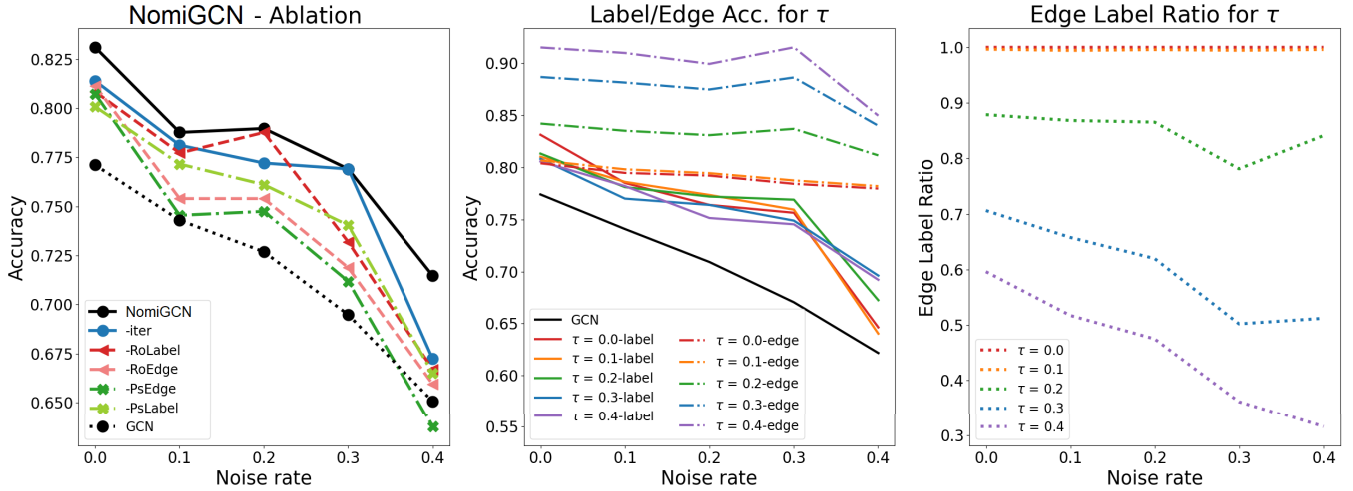


Fig. 2. (Left) results of ablation study. (Middle/Right) Hyperparameter analysis for threshold τ .

TABLE VI
ACCURACY OF NODE CLASSIFICATION ON PUBMED AND OGBN-ARXIV. (* MEANS SIGNIFICANT IMPROVEMENT BY WILCOXON SIGNED-RANK TEST)

	Pubmed						Ogbn-arxiv					
Noise Rate	0.0	0.1	0.2	0.3	0.4	$\uparrow(\%)$	0.0	0.1	0.2	0.3	0.4	$\uparrow(\%)$
GCN	0.8012	0.8036	0.8049	0.8046	0.7925		0.6049	0.6034	0.6008	0.5985	0.5936	
NomiGCN	0.8471	0.8377	0.8259	0.8137	0.7959	2.82*	0.6494	0.6405	0.6325	0.6247	0.6157	5.38*
GS	0.8184	0.8128	0.7970	0.7823	0.6892		0.6410	0.6390	0.6340	0.6313	0.6254	
NomiGS	0.8187	0.8143	0.8005	0.7865	0.6932	0.35*	0.6434	0.6417	0.6363	0.6327	0.6263	0.30*
GIN	0.7266	0.6344	0.5774	0.5169	0.4885		0.6233	0.6138	0.6032	0.5924	0.5770	
NomiGIN	0.7599	0.7083	0.6608	0.6160	0.5897	14.11*	0.6242	0.6150	0.6048	0.5943	0.5802	0.30*
GAT	0.8259	0.8227	0.8115	0.8069	0.7900		OOM					
NomiGAT	0.8269	0.8225	0.8114	0.8070	0.7947	0.14	OOM					
GCNII	0.8444	0.8417	0.8357	0.8227	0.8142		OOM					
NomiGCNII	0.8500	0.8431	0.8353	0.8229	0.8145	0.17	OOM					
LafAK-D	0.8284	0.8278	0.8218	0.8202	0.8021		0.6301	0.6343	0.6291	0.6249	0.6175	
NomiLafAK-D	0.8481	0.8381	0.8230	0.8195	0.8021	0.74*	0.6666	0.6583	0.6498	0.6425	0.6334	3.65*
$\uparrow(\%)$	2.25	2.91	2.93	3.47	3.73		3.80	3.08	2.76	2.60	1.75	

merely debias the loss; it also prunes unreliable edges and amplifies homophilous signal for message passing, thereby suppressing error propagation that pure loss correction cannot curb. These trends mirror the theory: NomiGNN's edge-label objective gives an *upper-bound control* on node-label loss, so reducing edge-loss tightens node-loss [Section IV-C, (13)], and its jointly corrected losses use T^{-1} and T_e^{-1} to neutralize node- and edge-label noise (Lemma 1 & Theorem 1). The ablation (Fig. 2-Left) further shows that removing robust edge optimization or pseudoedge labeling erodes the gains, confirming that structure-aware filtering, *not* loss correction alone, drives the robustness we observe at high noise.

2) *Why NomiGNN Beats Correct-GNN, and When It Does Not?*: Correct-GNN assumes the graph is a neutral conduit and fixes only the label-side bias via T^{-1} . In contrast, NomiGNN: 1) estimates T and induces T_e from T to correct edge supervision; 2) learns a structure-aware edge-consistency predictor f_e from node posteriors [see (10)–(12)]; and 3) expands reliable supervision with pseudo-edge labels plus iteration to refine T, T_e . This combination directly prevents noisy neighbors from dominating aggregation and, by the loss-bound coupling,

translates cleaner edges into lower node risk, precisely why NomiGNN outperforms across backbones and especially at high noise in Table VII. In the few low-noise cases where Correct-GNN is marginally better, the reasons are also principled: when labels are already clean (or nearly so) and the graph is very homophilous (e.g., Polblog): 1) the variance from estimating T and propagating it to T_e can slightly perturb training; 2) thresholded pseudo-edge labeling may over-prune rare but useful cross-class edges; and 3) the added multitask head and constraints act as extra regularization, which is beneficial under noise, but a tiny drag when noise is minimal and loss-correction already matches clean-loss optimization. These effects fade as noise rises, where edge-aware denoising becomes decisive and NomiGNN's advantage dominates the table.

C. Experiments on Heterophilic Graphs

To evaluate NomiGNN's effectiveness beyond homophilic settings, we conduct experiments on heterophilic graph datasets where connected nodes tend to have different class labels. These datasets, including Wisconsin, Chameleon, and

TABLE VII

ACCURACY OF NODE CLASSIFICATION FOR NOMIGNN VERSUS CORRECT-GNN BY VARYING THE NOISE RATE FROM 0.0 TO 0.9 ON CORA, CITESEER, POLBLOG, ACM, AND DBLP (FROM TOP TO BOTTOM)

Cora	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
GCN	0.7953	0.7570	0.7077	0.6463	0.5986	0.5220	0.4170	0.3032	0.2226	0.1566
Correct-GCN	0.8173	0.7791	0.7369	0.6843	0.6257	0.5267	0.4214	0.3067	0.2232	0.1626
NomiGCN	0.8262	0.8041	0.7659	0.7303	0.6741	0.5915	0.4871	0.3488	0.2551	0.1651
NRGNN	0.8308	0.8187	0.8005	0.7865	0.7588	0.6646	0.5316	0.4029	0.2973	0.1694
Correct-NRGNN	0.8348	0.8265	0.8089	0.7924	0.7673	0.7038	0.5667	0.4317	0.3122	0.1568
NomiNRGNN	0.8451	0.8231	0.8138	0.7918	0.7729	0.7296	0.5916	0.4540	0.3156	0.1661
RSIGN	0.8168	0.7955	0.7664	0.7338	0.6973	0.5809	0.4518	0.3285	0.2560	0.1731
Correct-RSIGN	0.7659	0.7411	0.7060	0.6603	0.6371	0.5879	0.4567	0.3348	0.2616	0.1750
NomiRSIGN	0.8233	0.8017	0.7824	0.7472	0.7171	0.6000	0.4703	0.3591	0.2609	0.1751
Citeseer	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
GCN	0.6616	0.6410	0.6076	0.5690	0.5105	0.4472	0.3749	0.2931	0.2377	0.1900
Correct-GCN	0.7005	0.6678	0.6389	0.5943	0.5692	0.4531	0.3829	0.2906	0.2398	0.1906
NomiGCN	0.7149	0.6782	0.6446	0.6020	0.5673	0.5235	0.4511	0.3412	0.2577	0.2227
NRGNN	0.7136	0.6999	0.6900	0.6799	0.6408	0.5614	0.4735	0.3444	0.2653	0.1759
Correct-NRGNN	0.7141	0.7027	0.6780	0.6672	0.6346	0.5292	0.4878	0.3475	0.2728	0.1764
NomiNRGNN	0.7393	0.7235	0.7006	0.6915	0.6553	0.6004	0.5031	0.3741	0.2876	0.2032
RSIGN	0.7188	0.7074	0.6917	0.6805	0.6450	0.5133	0.4399	0.3031	0.2433	0.1820
Correct-RSIGN	0.6864	0.6694	0.6563	0.6467	0.6117	0.5216	0.4442	0.3015	0.2474	0.1892
NomiRSIGN	0.7201	0.7175	0.7088	0.7017	0.6643	0.5722	0.5113	0.3442	0.2758	0.2133
Polblog	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
GCN	0.9501	0.9073	0.8417	0.7312	0.6669	0.5937	0.4888	0.4775	0.4558	0.4582
Correct-GCN	0.9423	0.9271	0.8718	0.7672	0.6953	0.5951	0.4892	0.4775	0.4545	0.4582
NomiGCN	0.9501	0.9507	0.9019	0.7699	0.7026	0.6135	0.4931	0.5024	0.5016	0.5085
NRGNN	0.9468	0.9233	0.9055	0.8052	0.7205	0.4775	0.3681	0.2209	0.3139	0.2822
Correct-NRGNN	0.9464	0.9181	0.9282	0.8520	0.7340	0.4755	0.4785	0.2045	0.2832	0.2372
NomiNRGNN	0.9518	0.9352	0.9204	0.8397	0.7559	0.6499	0.4899	0.4749	0.4775	0.4376
RSIGN	0.9024	0.8367	0.8171	0.7277	0.6604	0.6060	0.4987	0.3076	0.4930	0.4898
Correct-RSIGN	0.9528	0.9455	0.9284	0.8637	0.7465	0.6073	0.4954	0.5076	0.4930	0.4898
NomiRSIGN	0.9264	0.9386	0.9339	0.9276	0.8973	0.6619	0.5133	0.3538	0.5266	0.5259
ACM	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
GCN	0.9068	0.8949	0.8796	0.8390	0.7532	0.6787	0.5105	0.3662	0.3150	0.2774
Correct-GCN	0.9091	0.8953	0.8840	0.8405	0.7596	0.6892	0.5182	0.3669	0.3181	0.2799
NomiGCN	0.9166	0.9041	0.8828	0.8481	0.7661	0.7518	0.5717	0.4093	0.3528	0.3111
NRGNN	0.8873	0.8811	0.8802	0.8790	0.8406	0.8331	0.5940	0.4144	0.3413	0.3119
Correct-NRGNN	0.9102	0.9033	0.9020	0.8910	0.8589	0.8320	0.6146	0.4209	0.3497	0.3101
NomiNRGNN	0.9046	0.9061	0.9020	0.8874	0.8600	0.8593	0.6245	0.4349	0.3757	0.3266
RSIGN	0.9043	0.9008	0.8978	0.8804	0.8327	0.7708	0.5808	0.3850	0.3588	0.3533
Correct-RSIGN	0.9088	0.9026	0.9027	0.8879	0.8364	0.7775	0.5826	0.3897	0.3702	0.3539
NomiRSIGN	0.9058	0.9031	0.8982	0.8943	0.8876	0.8773	0.6077	0.5510	0.4228	0.4234
DBLP	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
GCN	0.7800	0.7400	0.7106	0.6488	0.5856	0.5502	0.4152	0.3377	0.2496	0.2202
Correct-GCN	0.7894	0.7489	0.7248	0.6479	0.6099	0.5589	0.4151	0.3406	0.2549	0.2202
NomiGCN	0.7922	0.7548	0.7283	0.6490	0.5949	0.5953	0.4378	0.3529	0.2633	0.2473
NRGNN	0.8129	0.8076	0.7889	0.7572	0.7281	0.6297	0.4950	0.3860	0.2575	0.2272
Correct-NRGNN	0.8143	0.8046	0.7895	0.7618	0.7226	0.6680	0.5246	0.3808	0.2548	0.2212
NomiNRGNN	0.8215	0.8154	0.7927	0.7612	0.7359	0.7114	0.5702	0.4178	0.2893	0.2441
RSIGN	0.8124	0.8028	0.7857	0.7540	0.7231	0.5345	0.4001	0.3295	0.2384	0.2117
Correct-RSIGN	0.7800	0.7482	0.7148	0.6733	0.6058	0.5375	0.4056	0.3334	0.2430	0.2121
NomiRSIGN	0.8191	0.8173	0.7939	0.7695	0.7453	0.5824	0.4687	0.3740	0.2976	0.2377

TABLE VIII

ACCURACY OF NODE CLASSIFICATION ON HETEROPHILIC DATASETS

Method	0.0	0.1	0.2	0.3	0.4	$\uparrow(\%)$
Chameleon						
NRGNN	0.3966	0.3829	0.3497	0.3337	0.3282	
NomiNRGNN	0.4036	0.3939	0.3757	0.3711	0.3603	7.71*
Wisconsin						
NRGNN	0.5119	0.4567	0.4224	0.4189	0.4139	
NomiNRGNN	0.5174	0.4706	0.4418	0.4383	0.4080	1.63
Squirrel						
NRGNN	0.2912	0.2785	0.2899	0.2832	0.2761	
NomiNRGNN	0.3115	0.3099	0.2972	0.2977	0.2904	5.47*

Squirrel, have edge label rates below 0.36 (see Table III), indicating low homophily. Since NRGNN consistently showed the best performance among baseline methods in our previous experiments, we focus on comparing NRGNN with NomiNRGNN on these challenging datasets. Table VIII shows the performance comparison across different noise rates. Despite the challenging nature of heterophilic graphs, NomiGNN consistently improves model performance, with gains on Chameleon (7.71%) and Squirrel (5.47%). Wisconsin shows a modest improvement of 1.63%. The improvement patterns on heterophilic graphs differ from those on homophilic graphs. While both show enhanced robustness against noise, the magnitude of improvement tends to be more consistent across noise levels in heterophilic settings. This suggests that NomiGNN's noise correction and edge classification mech-

anism provides valuable information even when connected nodes typically have different labels. These findings demonstrate that NomiGNN's benefits extend beyond the standard homophilic assumption, making it applicable to diverse real-world graph scenarios. NomiGNN helps distinguish between informative and disruptive edges regardless of whether the overall graph structure follows homophily patterns, showing our method's broad applicability.

D. Ablation Study

We aim to show the contribution of each proposed component in NomiGNN. In the study, we select GCN as the base GNN model, and apply five variants of our NomiGNN for node classification on Cora. The variants include: 1) **-iter**: NomiGNN without an iterative robust learning process. To clearly present the difference before and after removing the proposed components, the following cases also do not consider iterative learning; 2) **-RoLabel**: removing the robust label optimization [see (4)]; 3) **-RoEdge**: removing the robust edge label optimization [see (7)]; 4) **-PsEdge**: removing the edge label classification via pseudo edge labeling; and 5) **-PsLabel**: replacing pseudo edge labeling with pseudo node labeling. We show the performance comparison under different noise rates for the full NomiGNN, original GCN, and these 5 variants in Fig. 2 (Left).

The results demonstrate NomiGNN's robustness and superiority in handling node classification tasks under varying noise levels. The full NomiGNN consistently outperforms the base GCN model and its own variants, underlining the collective strength of its components. Key findings highlight the pivotal role of iterative learning in managing noise, with the **-iter** variant showing a significant performance dip. Moreover, the essential nature of robust label and edge optimizations is evident from the decreased accuracies of **-RoLabel** and **-RoEdge**. The comparison between **-PsEdge** and **-PsLabel** highlights the benefits of edge labeling. Pseudo edge labeling likely captures relationships or interactions between nodes more effectively than pseudo node labeling, leading to better accuracy. In essence, NomiGNN's resilience to noise and its high accuracy make it a compelling choice for practitioners dealing with real-world graph data.

E. Hyperparameter Analysis

We discuss how the choice of the threshold of pseudoedge labeling τ influences the performance of node classification in the presence of label noise. The high value of τ means more strict to decide the pseudo edge label with high predicted probability, and therefore the number of selected edge labels is small. It means that the choice of τ is a tradeoff between the pseudo edge labeling accuracy and the pseudoedge label number. To understand the difference in detail, we conduct the node classification with GCN as the base GNN on Cora under different noise rates, and separate three aspects including the accuracy of node classification, the label consistency with the selected pseudo labeled edges, and the number of the selected edges. In Fig. 2, two accuracy scores are shown in the middle

TABLE IX
COMPARISON OF EDGE LABEL RATE WITH GCN

Datasets	Edge Label (%)	GCN	$\uparrow$ (%)
Polblog	0.9058	0.9501	8.44
ACM	0.8208	0.9068	7.27
Cora	0.8041	0.7953	4.34
Pubmed	0.8024	0.8012	2.82
DBLP	0.7988	0.7800	1.03
CiteSeer	0.7364	0.6616	1.56
Ogbn-arxiv	0.6542	0.6049	5.38

subfigure in solid and dotted lines, and we represent the label edges by the edge label ratio (i.e., #pseudo-labeled edges / #edges in graph), in the right subfigure. Each line represents the accuracy scores under different noise rates with the given threshold.

For node classification, though different choices of the threshold cause inconsistent performances over noise rates, they all gain the robustness compared with the base GCN model. On average, we think that $\tau = 0.2$ (i.e., the green line) is a better choice. For the accuracy of pseudo-labeled edges, the line tendency looks to decrease in higher noise rates because the corruption of noise influences the performance of node classification as well as the perdition of pseudo edge labels. Differently, the decrease in the accuracy for edge label prediction is slower than the accuracy change of node classification. The reason is that predicting the edge labels has more tolerant than node classification, as discussed in Section IV-C1. Besides, the line of pseudoedge labeling ratio also declines as the noise rate increases. Such an outcome is similar to the tendency of pseudo edge labeling accuracy, which is influenced by noise label corruption. Thus, more noise would hurt the overall prediction probability, and then the possibly selected pseudo edge labels are fewer for the fixed τ . To sum up, although the threshold does not have a direct relation to the performance, NomiGNN still can boost the performances in different τ . The optimal choice of threshold is suggested to depend on the dataset and validation tuning.

F. Homophily Versus Edge Label Rate

We have compiled preliminary results from a performance comparison between NomiGCN and GCN across seven datasets, as detailed in Table IX. The datasets are organized by edge label rate (Column 2), alongside the observed percentage increase in accuracy that NomiGCN provides over GCN (Column 3). These findings reveal no apparent correlation between the edge label rate and the performance improvements offered by NomiGNN. For instance, despite Ogbn-arxiv having a relatively low edge label rate of 0.65, NomiGCN demonstrates a significant improvement of approximately 4.3%, outperforming DBLP which has a higher edge label rate of 0.8. The accuracy enhancements for Cora, PubMed, and DBLP, despite having comparable edge label rates, exhibit variations ranging from about 1.03% to 4.34%. This variability suggests that the architecture and mechanisms of NomiGNN play a critical role in its effectiveness across different graph structures, indicating that its performance is influenced by factors beyond merely the level of homophily.

TABLE X
AVERAGE NUMBER OF ITERATIONS BEFORE CONVERGENCE ON PUBMED AND DBLP FOR NOMIGAT

Noise Rate	Pubmed	DBLP
0.0	1.59	1.09
0.1	1.19	1.07
0.2	1.09	1.07
0.3	1.50	1.05
0.4	1.20	1.14

TABLE XI
IMPROVEMENT (%) BY INCORPORATING NOMIGNN INTO DIFFERENT GNN METHODS, SORTED BY DATASET SIZE

Datasets	size (#node)	GCN	GS	GIN	LafAK-D
Polblog	1222	8.44	3.75	12.39	2.04
CiteSeer	2110	1.56	15.72	2.39	0.74
ACM	2464	7.27	2.15	3.95	5.43
Cora	2485	4.34	17.97	1.90	5.54
DBLP	2591	1.03	1.39	9.48	1.88
Pubmed	19717	2.82	0.35	14.11	0.74
Ogbn-arxiv	169343	5.38	0.30	0.30	3.65

G. Scalability Study

The proposed NomiGNN introduces additional computational steps designed to enhance robustness against label noise, compared to the standard GNN training. While noise distribution estimation and edge label prediction add some overhead, this is mitigated by our design choices. Noise distribution estimation uses a $c \times c$ noise transition matrix to transform the loss function, where c is the number of class labels, resulting in minimal computational cost. Edge label prediction employs link prediction techniques, converging quickly, often within a few batches. The iterative noise estimation process adapts dynamically to graph size and label noise ratio, as supported by empirical results showing no direct correlation with increased computational time. We verify it by showing the average number of iterations of GAT on Pubmed and DBLP datasets in Table X. The larger graph and noise rate would not increase the computation of NomiGNN. Furthermore, we focus on moderately sized datasets to fine-tune and demonstrate the fundamental capabilities of NomiGNN in Table XI. The results indicate that while performance variations exist across different sizes, NomiGNN shows promising scalability. In short, NomiGNN can achieve scalability across varying graph sizes and noise rates.

H. Noise Study: More Noise Variants

In the previous parts, we follow existing studies to generate label noise [7], in which the noise is flipped from the original class to other classes uniformly. Though the assumption satisfied the real-world scenario like labeling typos, there are other noises like edge noise and nonuniform label distribution that may occur in the graph data. Therefore, besides the uniform label noise denoted as **+Label**, we investigate five additional noise types, as described below, and examine how they influence the model robustness of NomiGCN, compared to GCN.

- 1) **+Edge**: randomly add new edges with the inconsistent label as noisy edges to dilute the edge label rate.

TABLE XII
PERFORMANCE ON DIFFERENT TYPES OF LABEL NOISE

	GCN	NomiGCN	Edge label rate
+Label	0.7093	0.7792	0.8041
+Edge	0.6871	0.7736	0.6246
+Edge+Label	0.6071	0.6595	0.6246
+Adv_Edge	0.3058	0.3124	0.1812
+Dist_Label	0.6494	0.7621	0.8041
+RDist_Label	0.7284	0.7797	0.8041

- 2) **+Edge+Label**: combine the creation of inconsistent labels on end nodes of edges (**+Edge**) with the original uniform label flipping noise (**+Label**).
- 3) **+Adv_Edge**: adopt the adversarial attack to perturb edges for the target nodes. We follow the process [8] that randomly selects 15% nodes in the graph as targets and apply Nettack [42] for stronger corruption.
- 4) **+Dist_Label**: Since different class labels may have inconsistent performances during training, a model's performance can be affected by the imbalanced class distribution or some dependent classes. We consider to simulate such kinds of nonuniform label noise by replacing the uniform distribution during label flipping. Specifically, we first train a model to evaluate the node classification performance of each class label, and take the corresponding accuracy as class weight. Then we flip the node labels based on the normalized weights as probabilities. Thus, the noisy labels would be more likely to become the classes with higher weights. This means that each flip would result in a more imbalanced noisy label distribution. The noise would mislead the model to be overfitting in the dominant class.
- 5) **+RDist_Label**: Similar to **+Dist_Label** but reversing the weights, we flip to generate the noisy labels according to the misclassification rate of each class, which reflects the learning difficulty. In other words, the flipped noisy labels possibly turns out to be the classes with a lower weights, i.e., high misclassification rates.

We examine how the NomiGNN framework works under such five variants of label noise. NomiGCN is compared with GCN (as the baseline) for the task of node classification. This experiment is conducted on the Cora dataset. Besides, we set the noise rate 0.15 for **+Adv_Edge** and 0.2 for others. The performance results in terms of accuracy for GCN and NomiGCN, together with the corresponding edge label rate in various generated noisy graphs, are reported in Table XII.

Compared with uniformly adding edge and label noise (i.e., **+Label**, **+Edge**, and **+Edge+Label**), we discover the positive relationship between the performance and edge label rate, which points out the importance of the edge label rate for accurate node classification. This also verifies that the PLabel and loss correction mechanisms in NomiGCN enhance the robustness of node classification in the presence of noisy labels and edges. For **+Adv_Edge**, Nettack is effective in hurting the performance of node classification, i.e., the accuracy is dropped greatly. Because our NomiGCN incorporates edge label classification and PLabel at the same time, it still slightly outperforms GCN under graph adversarial attack.

To discuss **+Dist_Label** and **+RDist_Label**, we find that NomiGCN with these noise variants can still provide stable and robust performance due to the proposed mechanisms of noise distribution estimation and loss correction. Besides, we find that making label distribution more skewed to the dominated classes has higher label noise degree than **+Label** and **+RDist_Label** (versus **+RDist_Label**). We think that the learning of GNNs relies on feature aggregation, and therefore the performance damage of GNNs would get stronger if the supervision of a GNN model is misled by more dominated class labels through message passing. That said, while the influence of label noise becomes stronger due to incorrect dominate classes, the proposed NomiGNN can still maintain the performance of node classification. Such outcome shows the effectiveness of NomiGNN against noisy class imbalance. In short, these results exhibit the capability of NomiGNN, which enhances GNN robustness for node classification under different types of label noise.

VI. CONCLUSION AND DISCUSSION

Addressing the challenge of GNN robustness in node classification amid label noise, we present a novel robust GNN framework, NomiGNN. NomiGNN enhances the robust loss correction mechanism for GNNs, enabling it to accurately estimate noise distributions and refine the training loss accordingly. We innovatively derive edge labels from paired labels and meticulously structure the prediction process, capitalizing on label probabilities. This ensures optimal edge identification, fostering accurate node classification while mitigating the spread via adverse edges. Our NomiGNN is versatile, allowing integration with a variety of GNN backbones. By melding loss correction with multitask learning for node labels and edge labels, and leveraging pseudo edge labeling and iterative learning, we address issues of label scarcity and estimation inaccuracies. Our rigorous theoretical insights are complemented by extensive empirical evaluations on five real-world graphs, showcasing the robustness enhancements our NomiGNN brings to eight diverse GNNs. Looking ahead, with the surge of expansive graphs, especially in social domains, our research will pivot toward enhancing robustness on larger scales. Beyond the nine foundational GNNs, we are intrigued by the potential boundaries of NomiGNN applicability and aspire to integrate it with more intricate GNN architectures.

REFERENCES

- [1] L. A. Adamic and N. Glance, "The political blogosphere and the 2004 U.S. Election: Divided they blog," in *Proc. 3rd Int. Workshop Link Discovery*, Aug. 2005, pp. 36–43.
- [2] D. Bo, X. Wang, C. Shi, M. Zhu, E. Lu, and P. Cui, "Structural deep clustering network," in *Proc. Web Conf.*, Apr. 2020, pp. 1400–1410.
- [3] L. H. Chen et al., "Erase: Error-resilient representation learning on graphs for label noise tolerance," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 270–280.
- [4] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1725–1735.
- [5] Y. Cheng, C. Shan, Y. Shen, X. Li, S. Luo, and D. Li, "Resurrecting label propagation for graphs with heterophily and label noise," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 433–444.

- [6] M. Crawshaw, "Multi-task learning with deep neural networks: A survey," 2020, *arXiv:2009.09796*.
- [7] E. Dai, C. Aggarwal, and S. Wang, "NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 227–236.
- [8] E. Dai, W. Jin, H. Liu, and S. Wang, "Towards robust graph neural networks for noisy graphs with sparse labels," in *Proc. 15th ACM Int. Conf. Web Search Data Mining*, Feb. 2022, pp. 181–191.
- [9] E. Dai and S. Wang, "Towards self-explainable graph neural network," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2021, pp. 302–311.
- [10] A. P. Dawid and A. M. Skene, "Maximum likelihood estimation of observer error-rates using the EM algorithm," *Appl. Statist.*, vol. 28, no. 1, pp. 20–28, 1979.
- [11] X. Du et al., "Noise-robust graph learning by estimating and leveraging pairwise interactions," 2021, *arXiv:2106.07451*.
- [12] B. Frenay and M. Verleysen, "Classification in the presence of label noise: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 5, pp. 845–869, May 2014.
- [13] Q. Lu and L. Getoor, "Link-based classification," in *Proc. 20th Int. Conf. Int. Conf. Mach. Learn. (ICML)*, Washington, DC, USA: AAAI Press, 2003, pp. 496–503.
- [14] T. Gong et al., "A comparison of loss weighting strategies for multi task learning in deep neural networks," *IEEE Access*, vol. 7, pp. 141627–141632, 2019.
- [15] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NIPS*, 2017, pp. 1024–1034.
- [16] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 22118–22133.
- [17] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, and J. Tang, "Graph structure learning for robust graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 66–74.
- [18] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–14.
- [19] D.-H. Lee et al., "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2013, vol. 3, no. 2, p. 896.
- [20] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3538–3545.
- [21] Y. Li, J. Yin, and L. Chen, "Unified robust training for graph neural networks against label noise," in *Proc. Adv. Knowl. Discovery Data Mining: 25th Pacific-Asia Conf.*, 2021, pp. 528–540.
- [22] T. Liu and D. Tao, "Classification with noisy labels by importance reweighting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 3, pp. 447–461, Mar. 2016.
- [23] M. Marcus, B. Santorini, and M. A. Marcinkiewicz, "Building a large annotated corpus of english: The Penn treebank," *Comput. Linguistics*, vol. 19, no. 2, pp. 313–330, 1993.
- [24] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 1196–1204.
- [25] H. Ni, C. J. Jin, and T. Murata, "Learning graph neural networks with noisy labels," in *Proc. 2nd ICLR Learn. Ltd. Labeled Data (LLD) Workshop*, 2019.
- [26] G. Patrini, A. Rozza, A. K. Menon, R. Nock, and L. Qu, "Making deep neural networks robust to label noise: A loss correction approach," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1944–1952.
- [27] M. Pechenizkiy, A. Tsymbal, S. Puuronen, and O. Pechenizkiy, "Class noise and supervised learning in medical domains: The effect of feature extraction," in *Proc. 19th IEEE Symp. Comput.-Based Med. Syst. (CBMS)* 2006, pp. 708–713.
- [28] D. Sculley and G. V. Cormack, "Filtering email spam in the presence of noisy user feedback," in *Proc. CEAS*, California, 2007, p. 11.
- [29] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Learning from noisy labels with deep neural networks: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8135–8153, Nov. 2023.
- [30] A. Vahdat, "Toward robustness against label noise in training deep discriminative neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5596–5605.
- [31] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, 2018.
- [32] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, and J. Bailey, "Symmetric cross entropy for robust learning with noisy labels," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 322–330.
- [33] S. Wu et al., "Class2simi: A noise reduction perspective on learning with noisy labels," in *Proc. 38th Int. Conf. Mach. Learn.*, vol. 139, Jul. 2021, pp. 11285–11295.
- [34] Y. Wu et al., "Mitigating label noise on graphs via topological sample selection," in *Proc. 41st Int. Conf. Mach. Learn.*, vol. 235, 2024, pp. 53944–53972.
- [35] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [36] T. Xiao, T. Xia, Y. Yang, C. Huang, and X. Wang, "Learning from massive noisy labeled data for image classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 2691–2699.
- [37] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Represent.*, 2019.
- [38] J. Yuan, X. Luo, Y. Qin, Y. Zhao, W. Ju, and M. Zhang, "Learning on graphs under label noise," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2023, pp. 1–5.
- [39] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Commun. ACM*, vol. 64, no. 3, pp. 107–115, Mar. 2021.
- [40] M. Zhang, L. Hu, C. Shi, and X. Wang, "Adversarial label-flipping attack and defense for graph neural networks," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 791–800.
- [41] Z. Zhang and M. R. Sabuncu, "Generalized cross entropy loss for training deep neural networks with noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8792–8802.
- [42] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *Proc. Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 6246–6250.
- [43] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, "Geom-gcn: Geometric graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2020.



I. Chung Hsieh received the master's degree in science in statistical science from National Chung Cheng University, Chiayi, Taiwan, in 2017.

He is a Research Assistant at the Networked Artificial Intelligence Laboratory, Department of Computer Science and Information Engineering, National Cheng Kung University (NCKU), Tainan, Taiwan, from 2018. He had several papers published at top venues, including IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING (TKDE), IEEE ACCESS, and NeurIPS. His main research

interests include graph representation learning, graph neural networks, data science, machine learning, and deep learning.



Cheng-Te Li (Member, IEEE) received the Ph.D. degree from the Graduate Institute of Networking and Multimedia, National Taiwan University, Taipei, Taiwan, in 2013.

He is a Professor at the Department of Computer Science and Information Engineering, National Cheng Kung University (NCKU), Tainan, Taiwan. He has papers published at top venues, including KDD, WWW, ICDM, CIKM, SIGIR, IJCAI, ACL, EMNLP, NAACL, ACM-MM, TKDE, TIST, TNNLS, CIM, and CSUR. He leads the Networked

Artificial Intelligence Laboratory (NetAI Lab) at NCKU. His research targets at machine learning, data mining, social networks, recommender systems, and natural language processing.